

What are AI Agents?

An Introduction to the New Frontier



JOAKIM ABDINUR IUSUF

JUN 17, 2026

Introduction

For most of the last decade, the interesting question to ask about a model was “what can it produce?” You hand it an input — a question, a sentence to translate, a prompt to illustrate — and it hands back an output. A person then decides what to do with that output and what to ask next. The model is powerful, but passive.

Agents change the question. Instead of asking what a model can produce in a single turn, we ask what it can accomplish over many turns, on its own. You give an agent a goal, and it works out the steps to reach that goal, calling tools, reading the results, and adjusting course, without a human steering each move.

This post is my attempt to distill Google’s recent whitepaper *Introduction to Agents* (November 2025), into something you can read in one sitting. The goal is that by the end, you understand not just what an agent is, but why it’s built the way it is.

A one-line definition to start with: an agent is a language model running in a loop, given tools, wrapped in code that decides what to feed the model at each step. Everything below is an unpacking of that sentence.

So, what makes up an agent?

It helps to break an agent into four parts. The whitepaper uses a body analogy, which I'll borrow.

The model is the brain. This is the reasoning engine: a general-purpose, fine-tuned, or multimodal language model whose job is to look at the current situation and decide the next move. The model's quality sets a ceiling on how well the agent can plan and reason. However, the model never sees the whole world. The model only ever sees what the system around it places in its context window. The agent is the thing curating that view.

Tools are the hands. A model on its own can only generate text. Tools are what let it reach out and affect something: an API call, a database or vector-store query, a function that runs code. Tool-use follows a three-step pattern: (1) the system describes what a tool does, (2) the model decides to call it with specific arguments, and (3) the system runs it and feeds the result back into the model's context.

The orchestration layer is the nervous system. This is the code that runs the loop. It holds the agent's state, manages its memory, applies a reasoning strategy (for more, read about chain-of-thought and ReAct), and decides at each step whether the model should think more or act now. This is where most of the engineering lives, and it's the part that turns a clever model into a system that gets things done.

Deployment is the body. A prototype on your laptop is fine for learning, but an agent only becomes useful when it's hosted somewhere it can run continuously, be monitored and logged, scale under load, and be reached by users or by other agents through an API.

If there's a single idea to take from this section, it's that an agent is fundamentally a context-management system. At every step, it assembles a context window (system instructions, the user's request, the running history, any knowledge it has retrieved, the tools it could use, and the results of tools it has already called), runs the model on

that window, reads the output, and rebuilds the window for the next step. What we used to call prompt engineering is now context engineering, and at the scale of an agent, it stops being a nice-to-have and becomes the entire job.

The loop that makes it all work

Underneath every agent is a cycle that's often summarised as Think, Act, Observe. Spelled out a little more, it has five steps: the agent (1) receives a goal, (2) gathers the context it needs, (3) reasons about a plan, (4) takes one concrete action, then (5) observes the result and either finishes or loops back to plan the next step.

A worked example makes this less abstract. Suppose a user asks a support agent, "Where is my order #12345?" A naive system would call a single lookup function and return whatever it got. An agent does something more deliberate. First, it plans: "to answer this properly, I need to find the order, then get its carrier and tracking number, and check the carrier's live status, then write a clear reply." With that plan in mind, it acts, calling `find_order("12345")`, and observes the result: a record that happens to include the tracking number ZYX987. That observation re-enters its context, and the loop continues. It calls `get_shipping_status("ZYX987")`, observes "Out for Delivery," and only then composes its answer.

The thing worth noticing is that **each action's result becomes part of the input for the next decision**. The agent isn't following a fixed script; it's reacting to what it learns, one step at a time, until the original goal is met.

Five levels of agents

Not every agent is equally capable. A useful way to scope what you're building is to think in levels, each one adding a capability to the last.

Level 0 is the model alone. No tools, no memory, no connection to the outside world, just the reasoning engine answering from what it learned during training. It can be remarkably good within those walls: it will explain the offside rule and walk you through the history of the Swedish national team. But ask it whether Sweden won its World Cup opener last night against Tunisia in spectacular fashion and it has nothing to offer. That match happened after its training data was collected, so as far as the model is concerned, it never happened at all.

Level 1 connects the model to tools. Hand it a search tool and the same question becomes answerable. The model recognises that “did Sweden win last night?” needs live information, calls a search API with the right query, reads back a result (“Sweden won 5–1”), and folds that fact into its reply. The same mechanism underlies a call to a stock-price API or a retrieval-augmented lookup over your own documents. This is the threshold where a model stops being a passive oracle and becomes a functioning agent.

Level 2 adds strategic planning. Here the agent handles goals that take several dependent steps, and the key skill is **context engineering**: assembling exactly the right information for each step. Consider the request, “find a good coffee shop halfway between my office and my client’s office.” The agent first calls a maps tool to compute the midpoint (say it lands on a town called Enköping), and then, this is the interesting part, it constructs a brand-new query out of that result: coffee shops in Enköping, rated four stars or higher. Each step’s output shapes the next step’s input, which keeps the context focused and the model accurate.

Level 3 is a team of agents. Rather than build one all-knowing super-agent, you build several specialists and let them call each other as tools. Picture a “project manager” agent handed the goal “launch the Solaris headphones.” It doesn’t do the work itself; it delegates. Competitive pricing research goes to one agent, press-release drafts to another, the product page to a third, and it stitches the results together.

The structure mirrors how a human team divides labor, and each agent stays simpler and easier to maintain as a result.

Level 4 is self-evolving. At this frontier, an agent notices a gap in its own capabilities and fills it by creating a new tool, or even a new agent. Our project-manager agent might realize it needs to track social-media sentiment, find that no such tool exists on its team, and respond by invoking an agent-creator to build one on the fly. This is mostly research territory today, but it shows where the line is heading: from using a fixed set of resources to expanding them autonomously.



Figure 1. The five levels.

Building one: model, tools, and orchestration

To build an agent, start with the model. The instinct is to pick whatever sits on top of the benchmark leaderboard. That's usually a mistake. What matters is performance on *your* task: if the agent writes code, evaluate it on your codebase; if it reads insurance claims, test it on your document formats. Then weigh quality against latency and cost, because the best model is the one that balances all three for the job at hand. A common pattern is to route, using a frontier model for the hard

planning and a small, fast model for high-volume work like classifying intent or summarising text.

Tools come in two broad families. The first is retrieval, or grounding: giving the agent a way to look things up before it speaks. **Retrieval-augmented generation (RAG)** lets it query external knowledge held in vector databases or knowledge graphs, and natural-language-to-SQL tools let it answer analytic questions ("what were our top-selling products last quarter?") straight from a database. Grounding answers in retrieved facts is the *single most effective lever against hallucination*. The second family is action: wrapping APIs and functions so the agent can send an email, update a record, or run a generated script in a sandbox. This family also includes human-in-the-loop tools, which let the agent pause and ask a person to confirm before doing something irreversible (think "plan mode" in Claude Code, where the agent asks you to pick between different options or tell it to do something else).

Tying tools to the model is the job of function calling. A schema (the OpenAPI specification is the long-standing standard) tells the model what a tool is for, what arguments it expects, and what it returns, which is what lets the model emit a valid call and interpret the response. The **Model Context Protocol (MCP)** has become popular as a more convenient way to discover and connect to tools, and some models ship native tools (like built-in search) where the call happens inside the model invocation itself.

Orchestration is where you express intent. Your most powerful lever is the system prompt, effectively the agent's constitution. This is where you give it a role ("you are a support agent for Acme Corp"), constraints, an output format, a tone, and explicit guidance on when to reach for which tool. A few concrete examples in those instructions tend to do more than any amount of abstract description.

Memory lives here too. Short-term memory is the agent's scratchpad: the running sequence of action-and-observation pairs that tells the

model what just happened. Long-term memory is almost always implemented as another tool, a RAG store the agent can query to recall a user's preferences or how it handled a similar task weeks ago.

When tasks get large, single agents give way to multi-agent designs, and a handful of patterns recur. The **Coordinator** puts a manager agent in charge of routing sub-tasks to specialists and aggregating their answers. The **Sequential** pattern is similar to an assembly line, where one agent's output is the next one's input. **Iterative Refinement** pairs a generator with a critic (another language model) that scores its work and sends it back for another pass. And the **Human-in-the-Loop** pattern inserts a deliberate pause for a person's approval before a consequential action.

Making agents reliable

Here's the catch that separates a demo from a product. Traditional software testing rests on assertions like `output == expected`. That doesn't work when the output is probabilistic by design and the "right" answer is a paragraph that has to be correct, complete, free of things it shouldn't say, and appropriately worded. Evaluating that usually takes another model.

The discipline for managing this is sometimes called **AgentOps**, a natural extension of DevOps and MLOps for systems whose behaviour you can't fully predict. A few practices make it work. You start by defining success as business outcomes rather than raw correctness: goal-completion rate, user satisfaction, latency, cost per interaction, and downstream effects like conversion or retention. You evaluate quality with an "LM as judge," a strong model scoring outputs against a rubric (did it answer correctly? was it grounded? did it follow instructions?) over a golden dataset sampled from real interactions, with a domain expert validating the results. You gate deployments on metrics, running each new version against the full evaluation set and comparing it to production before rolling it out gradually with A/B tests.

When something does go wrong, you need to answer “why,” and that’s what traces are for. An OpenTelemetry trace records the agent’s entire trajectory — the exact prompt sent, the model’s reasoning, the tool it chose, the parameters it generated, and the raw observation that came back — so you can find the root cause instead of guessing. Finally, you treat human feedback as a gift rather than noise. A thumbs-down is a real edge case your evaluations missed; the mature move is to capture it, reproduce it, and convert it into a permanent test, so the same class of failure can’t quietly return.

Connecting agents to people and to each other

With humans, the usual interface is a UI. At its simplest, that’s a chatbot returning text; richer agents emit structured data like JSON to drive dynamic front-ends. The human-in-the-loop patterns (confirming an action, clarifying an ambiguous request, refining intent) are what keep a person meaningfully in control.

With other agents, the challenge is sharper. As an organisation builds more and more specialized agents, wiring them together with one-off custom integrations becomes a maintenance nightmare. The **Agent2Agent (A2A) protocol** is the proposed standard for this. It lets an agent publish an “Agent Card,” a small JSON file that advertises what the agent can do, where to reach it, and how to authenticate, which makes discovery uniform. Where MCP is built for transactional tool calls, A2A is built for one agent handing a problem to another, using asynchronous tasks that can stream updates over a long-running connection. This is the thing that makes the Level-3 team-of-specialists idea practical at scale.

Security: the trust trade-off

There's a tension you feel the moment you give an agent any real power. To be useful, it needs autonomy and tools: the ability to send emails, query databases, move money. But every capability you grant widens the blast radius if the agent does something it shouldn't, whether that's a rogue action or leaking sensitive data. And you can't simply trust the model's judgment, because that judgment can be hijacked by prompt injection.

The accepted answer is defense in depth, in two layers. The first is deterministic guardrails: hard-coded rules that sit outside the model's reasoning and can't be argued with, like blocking any purchase over \$100 or requiring explicit confirmation before an external call. The second is reasoning-based defense: training the model to resist attacks, and running small specialized "guard models" that inspect a proposed plan and flag risky or policy-violating steps before anything executes.

How agents learn and evolve

Agents deployed in the real world live in a moving environment: policies change, data formats shift, regulations update. Left untouched, an agent slowly "ages" and loses accuracy, and hand-patching a whole fleet of them doesn't scale. The better approach is to design agents that adapt on their own.

They learn from two sources. The first is **runtime experience**: the logs, traces, and memory that capture their successes, failures, and tool interactions, and above all the corrections that humans hand them along the way. The second is **external signals**: new documents, updated rules, and critiques from other agents. They turn that raw material into better behavior in two ways: by refining their own context (prompts, few-shot examples, what they retrieve from memory) and by optimizing or creating tools to close capability gaps.

Take a regulated industry like healthcare or the life sciences. Imagine an agent that generates reports that must comply with privacy rules like

GDPR, built as a small team. A Querying Agent pulls the raw data, a Reporting Agent drafts the report, and a Critiquing Agent checks that draft against the compliance rules it knows, escalating anything ambiguous to a human expert. Watching over all of it is a Learning Agent. When the human expert flags that, say, certain household statistics must be anonymised, the Learning Agent records that correction and generalises it into a reusable rule. The next time a similar report comes through, the Critiquing Agent applies the rule automatically. The system absorbs the feedback once and stops needing it, which is exactly how you'd want a teammate to learn.

What this looks like at the frontier

Three examples give a sense of how far these ideas already reach.

Google Co-Scientist acts as a virtual research collaborator. You hand it a research goal and ground it in chosen sources, and it spins up a whole ecosystem of agents to explore the problem. A supervisor agent plays project manager, planning the work and delegating to specialists, and the team runs for hours or even days, looping not only on the hypotheses it generates but on the way it judges and generates them. (Google Co-Scientist: <https://deepmind.google/blog/co-scientist-a-multi-agent-ai-partner-to-accelerate-research/>.)

AlphaEvolve discovers and optimizes algorithms by pairing the code-generation ability of Gemini models with an automated evaluator, in an evolutionary loop: generate candidate solutions, score them, and use the best as the seed for the next generation. It has already improved the efficiency of Google's data centers, found faster matrix-multiplication algorithms, and produced new solutions to open mathematical problems. It works best on exactly the class of problems where checking an answer is far easier than finding one. (AlphaEvolve: <https://deepmind.google/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/>.)

Ruflo is a good place to see these ideas in code rather than on slides. It's an open-source orchestration framework, built on top of Claude Code, that turns a single coding assistant into a coordinated swarm of specialized agents, with shared memory, task routing between agents, model selection by difficulty, and tool integration through MCP. It also learns over time, so it's an example of self-evolving agents. (RuFlo: github.com/ruvnet/ruflo.)

Final thought - the shift worth internalising

If you take one thing from all of this, let it be the change in the developer's role. When you build ordinary deterministic software, you're a bricklayer: you specify every step, and the program does exactly and only what you wrote. Building an agent is closer to directing. You set the scene with instructions, choose the cast of tools, supply the context, and then guide an autonomous actor toward a performance you can't fully script in advance. In my next article, I will write about this shift in software engineering, called "agentic engineering".

One thing that's worth remembering about agents is that the flexibility that makes language models so capable is the very thing that makes them unreliable, so the real work moves out of the prompt and into the system surrounding it: solid tool contracts, careful context management, resilient error handling, and constant evaluation. Get that engineering right, and an agent stops being a fancier kind of workflow automation and starts behaving like something new: a capable, adaptable member of the team.